INF384 · Material de consulta para el Examen 1

Capítulo	Tema	Sesiones
1	Ciclo de vida de desarrollo de software	1
2	Introducción a DevSecOps	2
3	Integración continua	3 y 4
4	Despliegue continuo	5 y 6
5	Automatización de calidad	7 y 8

Capítulo 1 · Ciclo de vida de desarrollo de software

Value stream mapping (VSM)

El VSM es el recorrido de una unidad de trabajo, desde la solicitud hasta el valor entregado, con tiempos medidos en cada paso y en cada espera. Viene de Lean manufacturing (Toyota) y es el diagnóstico base de The DevOps Handbook. No es un diagrama de arquitectura, un BPMN, un tablero de tareas ni un organigrama.

Medida	Definición	Observación
PT · tiempo de proceso	Tiempo en que alguien trabaja efectivamente en la unidad	Se mide con quien ejecuta, no con el reporte del sistema
LT · lead time del paso	Tiempo total desde que la unidad entra al paso hasta que sale	Incluye cola, interrupciones y esperas
%C&A · completo y correcto	Porcentaje de veces que el paso recibe trabajo utilizable sin devolverlo	Un %C&A bajo revela retrabajo invisible

Eficiencia de flujo = (Σ PT / Σ LT) × 100

En organizaciones sin intervención la eficiencia de flujo suele estar entre 1% y 15%. Un valor alto indica poco tiempo en cola; no indica equipos rápidos.

Cómo se construye un VSM

1. Definir el alcance y la unidad de trabajo (historia, defecto, solicitud de cambio).
2. Reunir a quienes hacen el trabajo, incluida la persona que aprueba.
3. Dibujar un bloque por actividad y un bloque por cola.
4. Capturar PT, LT y %C&A por paso, como rango: caso típico y peor caso.
5. Calcular la eficiencia de flujo y atacar primero la espera más larga.

Ejemplo resuelto (horas calendario)

Paso	PT	Espera	LT	%C&A
Refinamiento	1 h	1 d 1 h	1 d 2 h	85%
Desarrollo	1 h	4 h	5 h	90%
Revisión de código	1 h	1 d	1 d 1 h	75%
QA manual	2 h	3 d	3 d 2 h	60%
Comité de cambios	1 h	2 d	2 d 1 h	95%
Despliegue	2 h	2 d	2 d 2 h	80%
Total	8 h		9 d 13 h	

Eficiencia de flujo = 8 h / 229 h × 100 ≈ 3.5%.

Métricas DORA

El VSM es una foto puntual; las métricas DORA son el monitoreo continuo. Se extraen del repositorio y del sistema de despliegue. Son la base empírica de Accelerate (2018).

Métrica	Eje	Qué mide
Frecuencia de despliegue	Velocidad	Con qué frecuencia se lleva código a producción. Señal del tamaño de lote
Lead time para cambios	Velocidad	Tiempo desde el commit hasta que corre en producción

Métrica	Eje	Qué mide
Tasa de fallas en cambios	Estabilidad	Porcentaje de despliegues que degradan el servicio y exigen corrección, rollback o parche
Tiempo de restauración del servicio	Estabilidad	Tiempo para recuperar el servicio tras un despliegue fallido o una degradación

Métrica	Élite	Alto	Medio	Bajo
Frecuencia de despliegue	Bajo demanda, varias veces al día	Entre una vez al día y una por semana	Entre una vez por semana y una al mes	Menos de una vez al mes
Lead time para cambios	Menos de un día	Entre un día y una semana	Entre una semana y un mes	Más de un mes
Tiempo de restauración	Menos de una hora	Menos de un día	Entre un día y una semana	Más de una semana
Tasa de fallas en cambios	Baja	Baja	Media	Alta

- **VSM** responde por qué el flujo es lento y dónde intervenir. **DORA** responde si está mejorando.
- **Hallazgo central de Accelerate:** los equipos de alto desempeño puntúan mejor en las cuatro métricas a la vez. Velocidad y estabilidad se mueven juntas por lotes pequeños, rollback barato, retroalimentación corta y automatización obligada.
- Los reportes recientes llaman a la cuarta métrica failed deployment recovery time y agregan rework rate.

Capítulo 2 · Introducción a DevSecOps

Modelo CALMS

CALMS es un instrumento de diagnóstico organizacional: califica qué tan lejos está una organización de operar bajo DevSecOps. Las cinco dimensiones se sostienen entre sí.

Dimensión	Definición	Señales de que existe	Señales de que no existe
C · Culture	Responsabilidad compartida sobre el resultado en producción y seguridad psicológica para reportar problemas	Quien construye opera; incidentes sin culpables; seguridad participa en el diseño	El postmortem termina con un nombre; «eso lo ve infraestructura»; problemas conocidos que no se escriben
A · Automation	El camino del repositorio a producción se ejecuta sin intervención manual e idéntico cada vez	Repetibilidad, trazabilidad, retroalimentación en minutos, criterio humano liberado para lo que requiere juicio	Despliegue manual que depende de quién lo hace
L · Lean	Optimizar el flujo completo con lotes pequeños y límite de trabajo en curso	40 despliegues al mes con 4 cambios cada uno	Un despliegue mensual con 180 cambios; 12 tareas abiertas a la vez
M · Measurement	Decisiones con datos del propio flujo; las métricas pertenecen al equipo	DORA medida automáticamente; el equipo ve sus números; se mide también seguridad	Se miden líneas de código, tickets cerrados u horas
S · Sharing	El conocimiento y las soluciones circulan entre equipos	Plantillas de pipeline, postmortems públicos, documentación junto al código, rotación	Una sola persona sabe desplegar

- Automatizar un proceso malo produce un proceso malo más rápido y con menos supervisión.
- Automatizar sin medir produce velocidad ciega; medir sin cultura produce métricas manipuladas.
- Reducir el tamaño del lote aumenta los despliegues, y más despliegues reducen el riesgo total.

Estructura de equipos

Modelo	Descripción	Condición de uso
Silos separados	Dev, Ops y Seguridad independientes con traspasos formales	Punto de partida más común; no es un modelo objetivo
Capacidad embebida	Cada equipo de producto incluye conocimiento de operación y seguridad	Funciona hasta cierta escala; luego duplica esfuerzo
Equipo de plataforma	Construye plataforma y plantillas que los equipos consumen como autoservicio	Modelo dominante a partir de cierto tamaño
Equipo de confiabilidad (SRE)	Define objetivos de servicio y comparte la operación bajo condiciones acordadas	Requiere madurez previa en medición y automatización

Anti-patrón: el equipo DevOps como tercer silo. Se crea un área DevOps que escribe y mantiene todos los pipelines; los equipos abren tickets y el área se vuelve el nuevo cuello de botella. La alternativa: el equipo especializado provee autoservicio, plantillas y valores por defecto seguros; cada equipo de producto es dueño de su pipeline; el éxito se mide por la autonomía de los demás.

Capítulo 3 · Integración continua

Estrategias de ramificación

Criterio	GitFlow	GitHub Flow	Trunk Based
Vida de la rama	Semanas	Días	Horas
Frecuencia de integración	Baja	Media	Alta (diaria)
Exige pruebas automáticas	Deseable	Necesario	Indispensable
Exige feature flags	No	A veces	Sí

- GitFlow fue diseñado para software con versiones publicadas explícitamente. En un servicio web con despliegue continuo es la causa más frecuente de ramas de tres semanas.
- Trunk Based solo funciona con verificación automática rápida y confiable; sin ella, rompe la rama principal todos los días.
- La estrategia se elige por las capacidades del equipo y el contexto.

Feature flags. El código llega a producción desactivado; activarlo es una decisión de producto. Desacoplan desplegar de publicar y permiten revertir apagando la bandera, sin desplegar. Costos: duplican caminos de ejecución y se vuelven deuda si nadie las retira. Toda bandera nace con fecha de retiro.

Versionado semántico X.Y.Z y convención de commits

Componente	Cuándo se incrementa
MAYOR (X)	Cambio incompatible con la versión anterior
MENOR (Y)	Funcionalidad nueva compatible hacia atrás
PARCHE (Z)	Corrección compatible hacia atrás

La versión identifica de forma única el artefacto y permite responder qué corre en producción. Es una convención: depende de que el autor de la librería clasifique bien el cambio.

Con commits convencionales el pipeline calcula la versión: feat incrementa MENOR, fix incrementa PARCHE y "!" (cambio incompatible) incrementa MAYOR. Desde la versión 1.2.0, dos fix, un feat y un ci producen 1.3.0. La convención se verifica en el pipeline; una convención que no se verifica se degrada en semanas.

Gestión de librerías

Lock file. Registra la versión exacta y la huella criptográfica de cada dependencia, declarada y transitiva. Sin lock file, «cualquier versión 4.x» trae 4.2.1 hoy y 4.3.0 mañana: el mismo commit produce dos artefactos distintos.

1. El lock file se versiona junto al código.
2. El pipeline instala estrictamente desde el lock file (por ejemplo npm ci).
3. Actualizar versiones es un cambio explícito que se revisa y prueba.
4. Sin reproducibilidad, el análisis de un incidente es una conjetura.

Build

El artefacto se construye una sola vez en el pipeline y ese mismo binario recorre todos los ambientes. Reconstruir por ambiente permite que cambie una dependencia transitiva entre construcciones: lo probado deja de ser lo desplegado.

Principio	Significado
Artefacto inmutable	Si hace falta un cambio, se construye una versión nueva
Configuración externa	Credenciales y parámetros se inyectan al desplegar
Identificado por versión	Lleva su versión y la referencia del commit
Almacenado	Para producción se descarga el artefacto ya validado

Elementos de un job de build reproducible: versión fija del entorno de ejecución, instalación desde el lock file, caché de dependencias y artefacto nombrado con la referencia del commit.

Validación de calidad

Quality gate: condición declarada que el resultado del análisis debe cumplir para que el pipeline continúe. Es explícita y versionada, su resultado es binario y detiene el pipeline. Sin quality gate, el análisis es un reporte.

Diseño del umbral. Un umbral sobre todo el código de un proyecto con historia se termina desactivando. El umbral se aplica sobre el código nuevo, parte del valor actual, se sube por acuerdo y toda exclusión se escribe con su razón en el repositorio. El análisis necesita el historial completo (fetch-depth: 0) para distinguir código nuevo.

Validación de seguridad en integración continua

El mismo hallazgo cuesta más cuanto más tarde aparece: en el commit lo corrige el autor con el contexto fresco; en producción hay incidente, exposición y reconstrucción de qué versión estuvo expuesta.

Criterios para que un control entre a esta etapa: rápido (minutos), determinista, acotable al cambio y sin ambiente desplegado.

Control	Qué hace	¿Entra a CI?
Detección de secretos	Busca credenciales, tokens y llaves en el código y el historial	Sí
Análisis de composición (SCA)	Compara el árbol de dependencias contra bases de vulnerabilidades conocidas	Sí
Análisis estático de seguridad (SAST)	Busca patrones explotables en el código fuente sin ejecutarlo	Sí
Análisis de configuración e IaC	Evalúa plantillas de infraestructura contra reglas escritas como código	Sí
Análisis dinámico (DAST)	Ataca la aplicación desplegada y respondiendo	No: requiere ambiente desplegado

- **Secretos:** borrar la línea no resuelve; el valor sigue en el historial. La remediación es rotar la credencial. Son dos controles: sobre el cambio y sobre el historial.
- **Composición:** el lock file es requisito; la mayoría de hallazgos cae en transitivas; se corrige subiendo la dependencia declarada que trae el componente. Produce además el inventario del artefacto.
- **Severidad no es prioridad.** Priorizar exige alcanzabilidad, exposición y existencia de corrección.

Política del gate de seguridad

1. Detiene lo nuevo e inventaría lo heredado.
2. Umbral por tipo de control: un secreto detiene siempre; una dependencia de severidad media con parche puede registrarse y planificarse.
3. La condición se escribe antes del hallazgo y queda versionada.

La excepción tiene cuatro datos: identificador del hallazgo, razón, responsable y fecha de vencimiento. Al vencer, el pipeline vuelve a detenerse. Una credencial nunca se exceptúa: se retira, se limpia del historial y se rota. El número de excepciones vigentes es la métrica que indica si el umbral está bien calibrado.

Capítulo 4 · Despliegue continuo

Entrega continua y despliegue continuo

La diferencia es una compuerta manual antes de producción; el pipeline es el mismo. La mayoría de organizaciones opera en entrega continua. El despliegue continuo sin observabilidad ni reversión automática traslada el costo del error al usuario.

	Entrega continua	Despliegue continuo
Qué ocurre con el cambio validado	Queda listo para desplegarse en cualquier momento	Llega a producción sin intervención manual
Quién dispara producción	Una persona	La validación automática
Requisito adicional	—	Confianza en la validación, observabilidad y reversión automática

Gestión de artefactos

Un artefacto desplegable es una unidad versionada e inmutable con todo lo necesario para ejecutar el sistema, salvo la configuración del ambiente. Tipos: paquete de aplicación (JAR, WAR, bundle), imagen de contenedor y plantilla de infraestructura. Propiedades: versionado, inmutable, trazable al commit y autocontenido.

Promover es apuntar el ambiente siguiente al mismo objeto binario: nada se reconstruye. El artefacto acumula evidencia en cada ambiente. El registro guarda versión, commit de origen, ejecución del pipeline y resultado de las validaciones.

Referencia	Ejemplo	Qué garantiza
Tag móvil	servicio:latest	Nada: es un puntero que se reasigna
Tag de versión	servicio:1.4.0	Inmutable solo por convención del equipo
Digest	servicio@sha256:9f2c…a41b	Identifica exactamente un contenido

La respuesta a «qué corre en producción» es el digest. En el Laboratorio 3: tag del commit (sha), versión y tag de ambiente prd apuntan al mismo manifiesto; el despliegue se hace por digest.

Imagen de contenedor

Mala práctica	Efecto	Práctica correcta
FROM node:latest	Dos builds del mismo commit producen imágenes distintas	Versión fija y base ligera (node:20.11-alpine)
COPY . .	Entran .git y .env; la caché se invalida en cada cambio	Copiar manifiesto y lock file primero; luego el código; .dockerignore
RUN npm install	Resuelve versiones nuevas en cada build	RUN npm ci desde el lock file
ENV DB_PASSWORD=…	El secreto queda en una capa, visible con docker history	Inyectar el secreto al desplegar
Herramientas en la imagen final	curl, vim, git amplían la superficie de ataque	Construcción en dos etapas: build y runtime
Sin USER	El proceso corre como root	Usuario sin privilegios declarado

Configuración por ambiente

Dentro del artefacto	Inyectado en el despliegue	Fuera del repositorio y del artefacto
Código construido, dependencias resueltas, valores por defecto, esquema de configuración	URLs de servicios, conexión a bases de datos, réplicas y límites, nivel de log, feature flags, identificador del ambiente	Contraseñas, llaves, tokens, certificados privados

Un valor por ambiente dentro del artefacto obliga a construir por ambiente y destruye la promoción.

Infraestructura como código (IaC)

Estado. Registra la correspondencia entre recursos declarados y reales. Se guarda remoto y con bloqueo, nunca en local, y se protege como secreto porque puede contener credenciales.

Plan y apply. El plan calcula la diferencia entre código, estado e infraestructura real; el apply la ejecuta.

1. El plan lo lee alguien más antes de aplicar.

2. Lo que se destruye importa más que lo que se crea: un replace (delete + create) de una base de datos se ve casi igual que un cambio de etiqueta.
3. El plan se calcula en el pull request y se aplica desde la rama principal.
4. Aplicar dos veces el mismo código produce una diferencia vacía.

Desviación (drift). Un cambio manual en la consola deja el código describiendo un ambiente que ya no existe; el siguiente plan propone revertirlo. Se detecta con un plan periódico sin aplicar y se cierra trasladando el cambio al código.

Pipeline de IaC: validar sintaxis (init, validate, fmt) → análisis de políticas sobre la declaración → plan publicado en el pull request → compuerta sobre el plan → apply del plan aprobado desde main en ambiente protegido.

Estrategias de despliegue

Desplegar (llevar el artefacto al ambiente; riesgo de disponibilidad) y publicar (exponer la funcionalidad; decisión de producto) son eventos separables. Una feature flag apaga la funcionalidad en segundos sin pipeline; revertir un despliegue toma minutos y requiere pipeline.

Criterio	Reemplazo total	Progresivo (rolling)	Dos ambientes (blue-green)	Canario
Indisponibilidad	Sí	No	No	No
Convivencia de versiones	No	Sí	Durante la transición	Sí, prolongada
Costo de infraestructura	Bajo	Bajo	Alto temporal	Medio
Velocidad de reversión	Lenta	Media	Inmediata	Inmediata
Exige observabilidad	Deseable	Deseable	Necesaria	Indispensable
Usuarios expuestos al fallo	Todos	Parte creciente	Todos tras conmutar	Porcentaje acotado

- **Blue-green:** la versión nueva se verifica con tráfico cero y el conmutador cambia en un instante. Los dos ambientes comparten la base de datos.
- **Canario:** 5% → 25% → 50% → 100% con una condición de avance (tasa de error, latencia, saturación) escrita antes de empezar y reversión automática. Con poco tráfico el 5% no da muestras suficientes.
- La estrategia se elige por el costo de un fallo en ese servicio y la capacidad real de detectarlo.

El esquema de datos es el límite real de toda reversión. Patrón expand-contract: (1) expandir agregando la estructura nueva; (2) escribir en ambas y migrar datos; (3) trasladar la lectura; (4) eliminar la estructura antigua en un despliegue propio. Toda migración se diseña compatible con la versión anterior en ejecución.

¿Revertir o corregir? Revertir despliega el artefacto anterior ya validado; es la opción por defecto en un incidente. Corregir hacia adelante exige un pipeline rápido; es la única opción cuando el esquema ya cambió. La decisión se define antes del incidente y la reversión se ensaya.

Flujo de aprobaciones

Lo que una aprobación resuelve	Lo que no resuelve
Segregación de funciones: quien escribe no autoriza	No detecta defectos
Registro auditable de quién decidió, cuándo y sobre qué versión	No reduce el riesgo del cambio; lo acumula con otros
Coordinación con el negocio por fecha o ventana	No sustituye una validación automática ausente

Accelerate reporta que la aprobación de un comité externo al equipo no mejora la estabilidad y alarga el tiempo de entrega. La alternativa es revisión por pares, validación automática y despliegue reversible. En contextos regulados la aprobación se conserva y el pipeline produce la evidencia.

Ambiente protegido: revisores requeridos, espera obligatoria, restricción de rama (solo main despliega a producción) y secretos por ambiente. La protección vive en la plataforma, fuera del archivo del pipeline. Quien aprueba necesita ver versión, diferencia y resultado de validaciones.

Credenciales del pipeline. Una llave estática guardada como secreto vive hasta que alguien la rota, se copia y suele tener alcance excesivo. La identidad federada (OIDC) entrega credenciales que expiran en minutos para un repositorio y rama específicos. El permiso se acota a lo que el trabajo necesita.

Pasos de un job de despliegue: compuerta de aprobación → credenciales de corta duración → desplegar por digest → esperar estabilidad → prueba de humo → revertir si falla.

Observabilidad y objetivos de servicio

Verificación del despliegue: desplegar → esperar estabilidad → prueba de humo → ventana de observación con duración, métrica y umbral declarados → confirmar o revertir sin consultar. Los despliegues fallidos alimentan la tasa de fallas en cambios.

Concepto	Definición
SLI · indicador	Medición de la experiencia del usuario, por ejemplo proporción de peticiones bajo un tiempo dado
SLO · objetivo	Valor acordado para el indicador en una ventana; un objetivo de 100% no tiene costo asociado
SLA · acuerdo	Compromiso con el cliente, con consecuencias si se incumple
Presupuesto de error	Diferencia entre el objetivo y la perfección; se gasta desplegando

Regla de consumo: mientras queda presupuesto, el equipo despliega a su ritmo; si se agota, la prioridad pasa a estabilidad. Con un SLO de 99.9% mensual (30 días), el presupuesto es 0.1% × 43 200 min ≈ 43 minutos.

Capítulo 5 · Automatización de calidad

La cartera de pruebas

Pirámide de pruebas: muchas unitarias en la base, menos de integración, pocas end to end en la punta. La proporción se decide por el costo de detectar cada defecto.

Triángulo invertido (cono de helado): regresión manual y suites de interfaz dominan; las unitarias existen solo para el umbral de cobertura. Produce lotes grandes porque probar cuesta mucho, certificación como etapa más larga del flujo y retrabajo tardío.

Presupuesto de tiempo (cartera de ejemplo)

Nivel	Cantidad	Tiempo c/u	Total secuencial
Unitarias	2 000	5 ms	10 s
Integración	150	2 s	5 min
End to end	40	45 s	30 min
Suite completa			≈ 35 min

Un objetivo frecuente es validar el commit en menos de 10 minutos. El paralelismo compra tiempo con cómputo (4 ejecutores bajan las end to end a unos 8 minutos) y no toda la suite corre en cada commit.

Defecto	Unitaria	Integración	End to end
Regla de negocio mal calculada	Detecta	Detecta tarde	Detecta tarde
Consulta SQL o migración incompatible	No detecta (el doble la oculta)	Detecta	Detecta tarde
Cambio de formato en la respuesta de una API	Parcial	Detecta	Detecta
Permisos, variables de entorno, configuración	No detecta	Parcial	Detecta
Recorrido roto entre varios servicios	No detecta	No detecta	Detecta

Cada nivel se justifica por los defectos que los inferiores no pueden ver.

Cobertura

Mide qué líneas y ramas ejecutó la suite; no mide si alguna aserción verificó el resultado. Una prueba sin aserciones puede dar 100% de cobertura. Funciona como piso del código nuevo. Las pruebas de mutación (cambiar ">" por ">=" y comprobar que alguna prueba falle) miden la verificación.

Pruebas de integración

Alcance	Descripción	Dónde corre
Acotada	Un componente propio contra una dependencia real y efímera	En el runner, en segundos

Alcance	Descripción	Dónde corre
Amplia	Varios servicios desplegados juntos	Ambiente compartido; hereda sus esperas

La integración acotada detecta lo que los dobles ocultan: SQL, migraciones, serialización, tiempos de espera. Patrón: la base de datos se declara como servicio del job, nace y muere con él, el job espera su chequeo de salud, se aplican las migraciones del repositorio y las pruebas de integración corren en un job propio.

Pruebas end to end

Qué recorridos merecen una: los que generan ingreso o cumplen una obligación; los que cruzan varios servicios; los que tuvieron incidentes en producción; un conjunto acotado y revisado periódicamente.

Pruebas sobre el ambiente desplegado

Etapa	Pruebas
Integración continua	Unitarias e integración acotada; sin ambiente
Publicar	Imagen construida una vez e identificada por digest
Certificación	Despliegue del digest, humo y end to end críticas
Promover	El mismo digest al ambiente protegido con aprobación
Producción	Humo, ventana de observación, verificación sintética

La evidencia de certificación vale para producción solo si se promueve exactamente el digest probado.

- **Prueba de humo:** pocas verificaciones justo después de desplegar; si falla, dispara la reversión.
- **Verificación sintética:** un recorrido crítico cada pocos minutos contra producción, con o sin despliegue, con datos marcados como de prueba. Su resultado es una señal de observabilidad.

Orquestación y política de compuertas

Lo rápido y lo que más falla corre primero; lo independiente corre en paralelo (unitarias e integración); las end to end se reparten entre ejecutores (matrix). El reporte de pruebas se sube aunque la prueba falle (if: always()). Promover usa el mismo digest y el ambiente protegido.

Resultado	Efecto	Dónde actúa
Falla unitaria o de integración	Detiene	La publicación del artefacto
Cobertura del código nuevo bajo el umbral	Detiene	La publicación (quality gate)
Falla un recorrido end to end crítico	Detiene	La promoción a producción
Falla una prueba en cuarentena	Informa	Registro con responsable y vencimiento
Falla la prueba de humo en producción	Revierte	Vuelve el digest anterior

La política se escribe en el repositorio antes de la primera falla.

Anexo · NIST SP 800-218 (SSDF) v1.1

El SSDF es un marco del National Institute of Standards and Technology (2022). Describe prácticas para reducir vulnerabilidades en el software que una organización produce. No prescribe herramientas: cada práctica indica qué resultado se espera, y la organización decide cómo lograrlo. Agrupa 19 prácticas en cuatro familias.

PO · Preparar la organización

Práctica	Qué pide
PO.1	Definir los requisitos de seguridad que debe cumplir el software y el proceso que lo produce
PO.2	Asignar roles y responsabilidades de seguridad en el ciclo de desarrollo
PO.3	Implementar herramientas de soporte que automaticen las prácticas de seguridad
PO.4	Definir criterios para verificar la seguridad del software y usarlos en cada entrega
PO.5	Mantener seguros los ambientes donde se desarrolla y se construye el software

PS · Proteger el software

Práctica	Qué pide
PS.1	Proteger todo el código contra acceso no autorizado y alteraciones
PS.2	Dar un mecanismo para verificar la integridad de cada versión publicada
PS.3	Archivar y proteger cada versión publicada y su información de origen

PW · Producir software bien protegido

Práctica	Qué pide
PW.1	Diseñar el software para cumplir los requisitos de seguridad y mitigar riesgos
PW.2	Revisar el diseño para verificar que cumple esos requisitos
PW.4	Reutilizar software existente y bien protegido cuando sea posible, y verificar los componentes de terceros
PW.5	Escribir código siguiendo prácticas de codificación segura
PW.6	Configurar la compilación y el proceso de construcción para mejorar la seguridad del ejecutable
PW.7	Revisar o analizar el código legible para encontrar vulnerabilidades
PW.8	Probar el código ejecutable para encontrar vulnerabilidades
PW.9	Configurar el software para que sea seguro por defecto

En la versión 1.1, la antigua PW.3 se integró en PW.4; por eso la numeración salta de PW.2 a PW.4.

RV · Responder a vulnerabilidades

Práctica	Qué pide
RV.1	Identificar y confirmar vulnerabilidades de forma continua
RV.2	Evaluar, priorizar y corregir las vulnerabilidades
RV.3	Analizar las vulnerabilidades para encontrar su causa raíz y evitar que se repitan